PetCare Manager

Documentacao da Aplicacao

Projeto de Pos-Graduacao em Servicos de Software

Stack: Python 3.11 + FastAPI | React 18 + Vite | PostgreSQL 15
Infraestrutura: Docker Compose (3 servicos)

Versao: 1.0.0

1. Visao Geral

O PetCare Manager e uma aplicacao web completa para gestao de animais de estimacao, permitindo o cadastro de pets, controle vacinal com alertas, registro de atividades fisicas e passeios, alem de um guia de cuidados especificos por raca.

A aplicacao foi desenvolvida como projeto de pos-graduacao em Servicos de Software, utilizando uma arquitetura moderna baseada em microsservicos com Docker Compose.

Funcionalidades

- CRUD completo de Pets com soft delete e busca por nome/raca
- Cadastro de 14 racas pre-carregadas (caes e gatos)
- Controle vacinal com alertas de vencimento (janela de 30 dias)
- Registro de atividades fisicas com sugestoes por nivel da raca
- Registro de passeios com sugestoes de locais por porte do pet
- Guia de cuidados por raca (70 cuidados: alimentacao, higiene, saude, etc.)
- Dashboard com estatisticas e alertas globais
- API REST documentada com Swagger automatico
- 52 testes automatizados cobrindo todos os endpoints

2. Arquitetura

A aplicacao segue uma arquitetura de 3 camadas orquestrada via Docker Compose:

Stack Tecnologica

Backend: Python 3.11 + FastAPI 0.111 + SQLAlchemy 2.0 + Pydantic v2

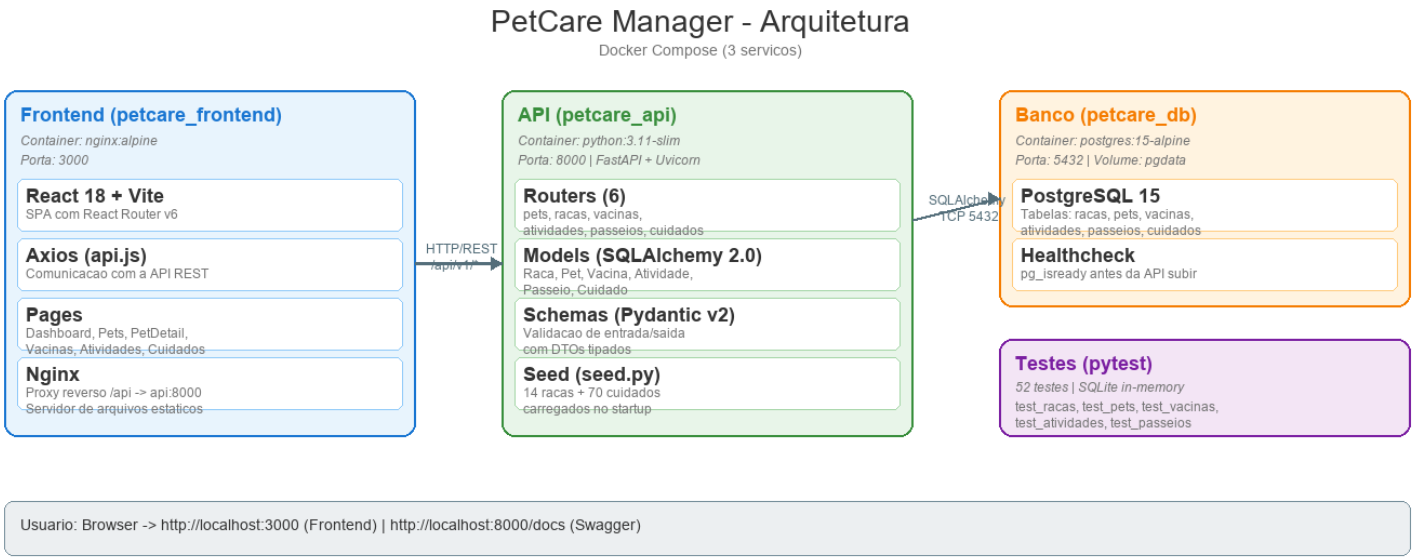
Frontend: React 18 + Vite 5.2 + React Router v6 + Axios + date-fns

Banco de Dados: PostgreSQL 15 (Alpine)

Servidor Web: Nginx Alpine (proxy reverso + SPA)

Testes: pytest + SQLite in-memory + httpx (TestClient)

Diagrama de Arquitetura



Funcionalidades Principais

- **Gestao de Pets**
  CRUD completo com soft delete. Cadastro com raca, peso, microchip, observacoes.
- **Controle Vacinal**
  Registro de vacinas aplicadas com alertas de vencimento (30 dias). Dashboard de pendentes.
- **Atividades & Passeios**
  Registro unificado. Sugestoes de atividades por nivel da raca e passeios por porte.
- **Guia de Cuidados**
  70 cuidados pre-cadastrados (5 por raca). Categorias: alimentacao, higiene, saude, comportamento, exercicio.
- **Dashboard**
  Visao geral com total de pets, vacinas pendentes e acesso rapido.
- **API REST**
  Endpoints em /api/v1/ com documentacao automatica Swagger em /docs

Fluxo de Requisicao

1. Usuario acessa http://localhost:3000 no navegador
2. Nginx serve o SPA React (arquivos estaticos)
3. React faz chamadas HTTP via Axios para /api/v1/*
4. Nginx faz proxy reverso para http://api:8000
5. FastAPI processa a requisicao, valida com Pydantic
6. SQLAlchemy executa a query no PostgreSQL
7. Resposta JSON retorna ao frontend

3. Estrutura do Projeto

```
petcare/
|-- docker-compose.yml          # Orquestra 3 servicos
|-- backend/
|   |-- Dockerfile              # Python 3.11-slim + curl
|   |-- requirements.txt        # Dependencias + pytest
|   |-- pytest.ini              # Configuracao dos testes
|   |-- app/
|   |   |-- main.py             # FastAPI app + CORS + lifespan
|   |   |-- database.py         # Engine SQLAlchemy + get_db
|   |   |-- seed.py             # Seed: 14 racas + 70 cuidados
|   |   |-- models/models.py    # 6 modelos + enums
|   |   |-- schemas/schemas.py  # DTOs Pydantic
|   |   |-- routers/            # 6 routers REST
|   |-- tests/                  # 52 testes automatizados
|       |-- conftest.py         # Fixtures (SQLite in-memory)
|       |-- test_racas.py       # 8 testes
|       |-- test_pets.py        # 11 testes
|       |-- test_vacinas.py     # 12 testes
|       |-- test_atividades.py  # 10 testes
|       |-- test_passeios.py    # 11 testes
|-- frontend/
    |-- Dockerfile              # Node 20 build + Nginx
    |-- nginx.conf              # Proxy + SPA + gzip
    |-- src/
        |-- App.jsx             # Shell + Sidebar + Rotas
        |-- services/api.js     # Axios centralizado
        |-- pages/              # 6 paginas React
```

4. Modelos de Dados

Raca

id, nome (unique), especie (enum: cao/gato/outro), porte (enum: pequeno/medio/grande), expectativa_vida_anos, descricao, nivel_atividade, created_at

Pet

id, nome, raca_id (FK), data_nascimento, sexo (enum: macho/femea), peso_kg, cor, microchip (unique), foto_url, observacoes, ativo (soft delete), created_at, updated_at

Vacina

id, pet_id (FK), nome, data_aplicacao, proxima_dose, veterinario, clinica, lote, status (enum: pendente/aplicada/atrasada), observacoes, created_at

Atividade

id, pet_id (FK), tipo, data, duracao_minutos, distancia_km, intensidade, observacoes, created_at

Passeio

id, pet_id (FK), local, data, duracao_minutos, avaliacao (1-5), observacoes, created_at

Cuidado

id, raca_id (FK), categoria, titulo, descricao, frequencia, prioridade

5. Endpoints da API

Base URL: /api/v1 | Documentacao interativa: /docs (Swagger)

Pets

```
GET    /pets/?ativo=true     Listar pets (ativos/inativos)
POST   /pets/               Criar pet
GET    /pets/{id}            Buscar pet por ID
PUT    /pets/{id}            Atualizar pet
DELETE /pets/{id}            Soft delete do pet
GET    /pets/{id}/dashboard  Dashboard do pet
```

Racas

```
GET    /racas/?especie=cao   Listar racas (filtro por especie)
POST   /racas/              Criar raca
GET    /racas/{id}           Buscar raca
PUT    /racas/{id}           Atualizar raca
```

Vacinas

```
GET    /vacinas/pet/{id}     Vacinas de um pet
GET    /vacinas/pendentes    Vacinas vencidas/proximas (30d)
POST   /vacinas/            Registrar vacina
PUT    /vacinas/{id}         Atualizar vacina
DELETE /vacinas/{id}         Remover vacina
```

Atividades

```
GET    /atividades/pet/{id}       Atividades de um pet
GET    /atividades/sugestoes/{id}  Sugestoes por nivel
POST   /atividades/               Registrar atividade
DELETE /atividades/{id}            Remover atividade
```

Passeios

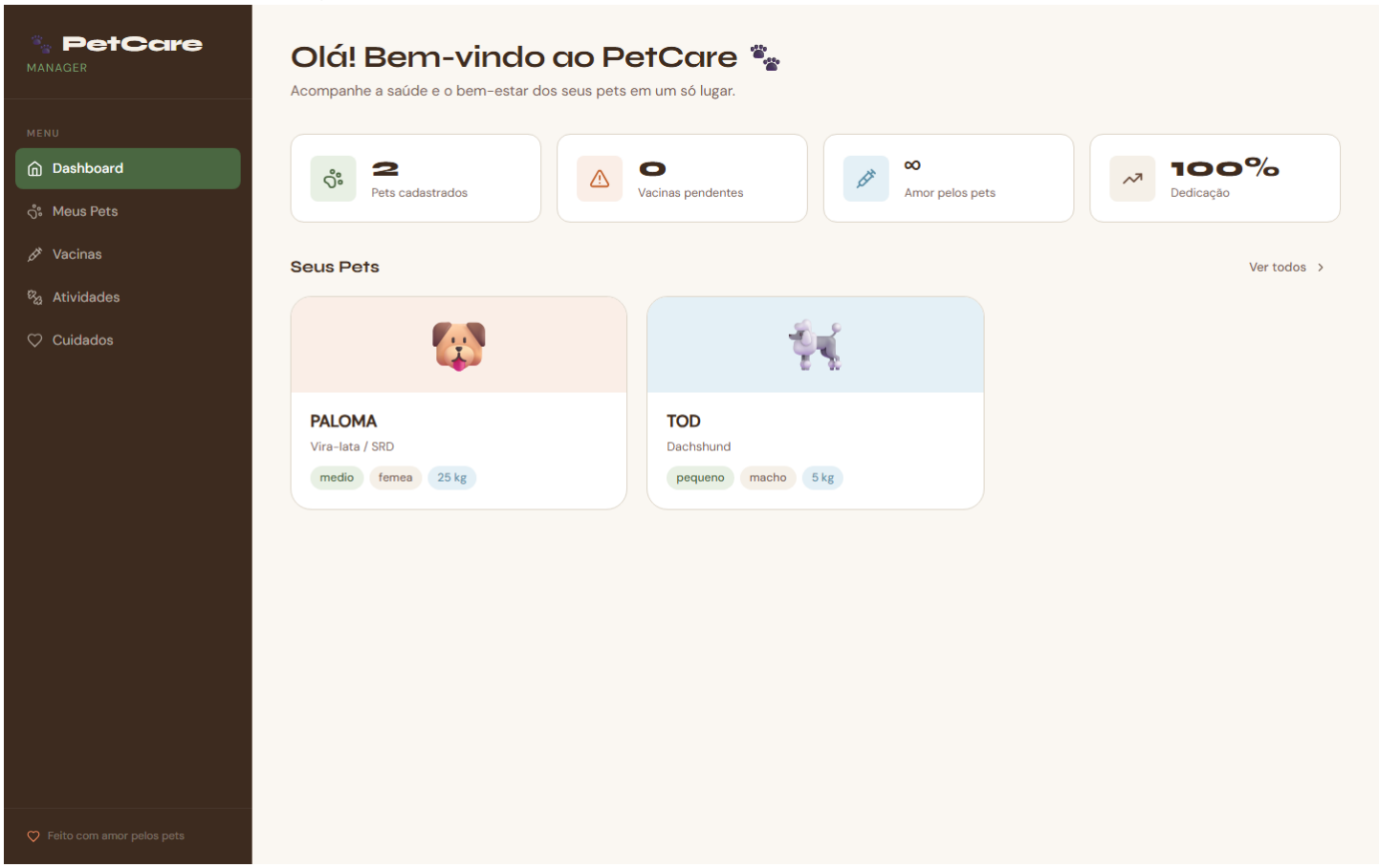
```
GET    /passeios/pet/{id}       Passeios de um pet
GET    /passeios/sugestoes/{id}  Sugestoes por porte
POST   /passeios/               Registrar passeio
DELETE /passeios/{id}            Remover passeio
```

Cuidados

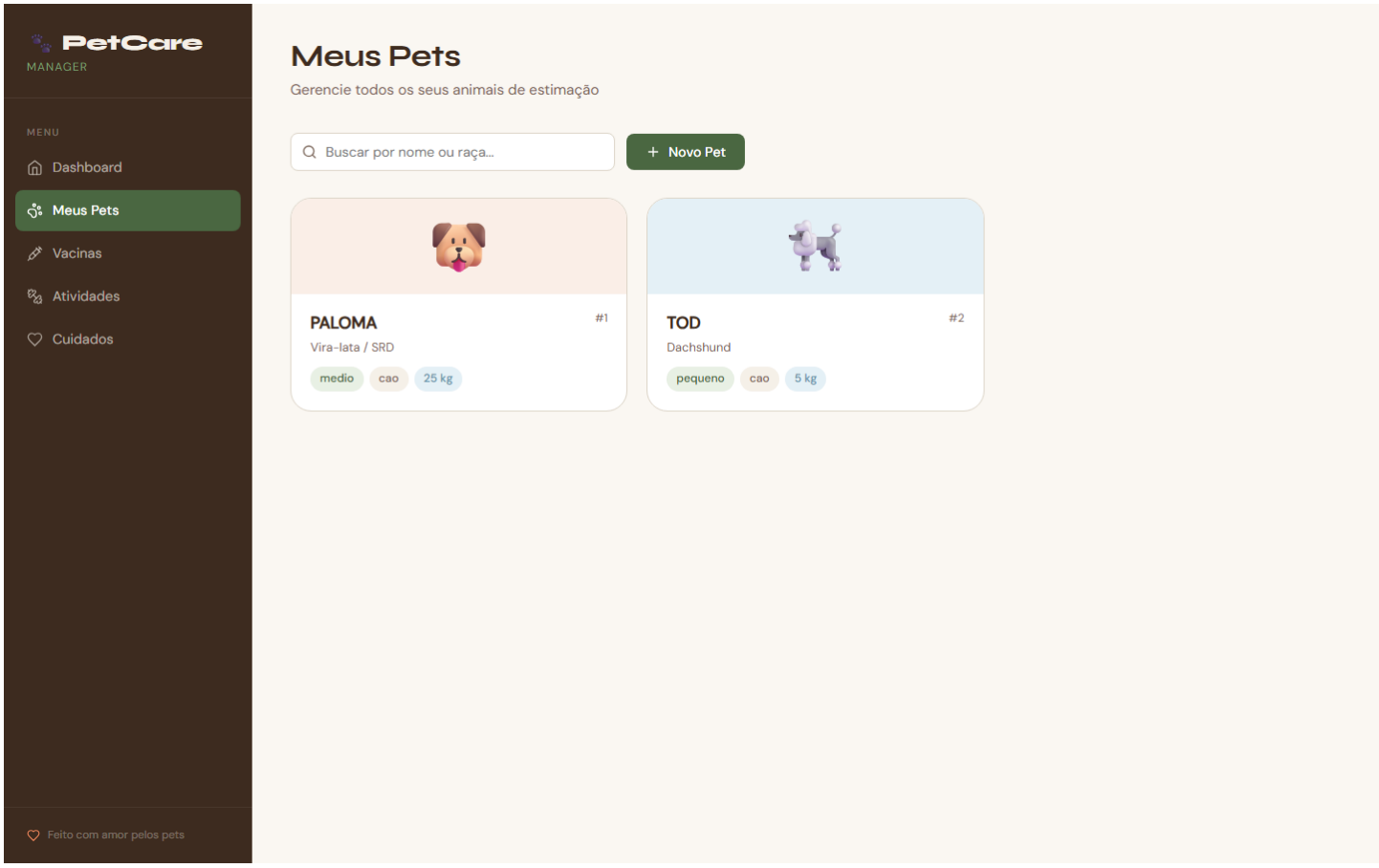
```
GET    /cuidados/raca/{id}  Cuidados de uma raca
POST   /cuidados/           Criar cuidado
DELETE /cuidados/{id}        Remover cuidado
```

6. Telas da Aplicacao

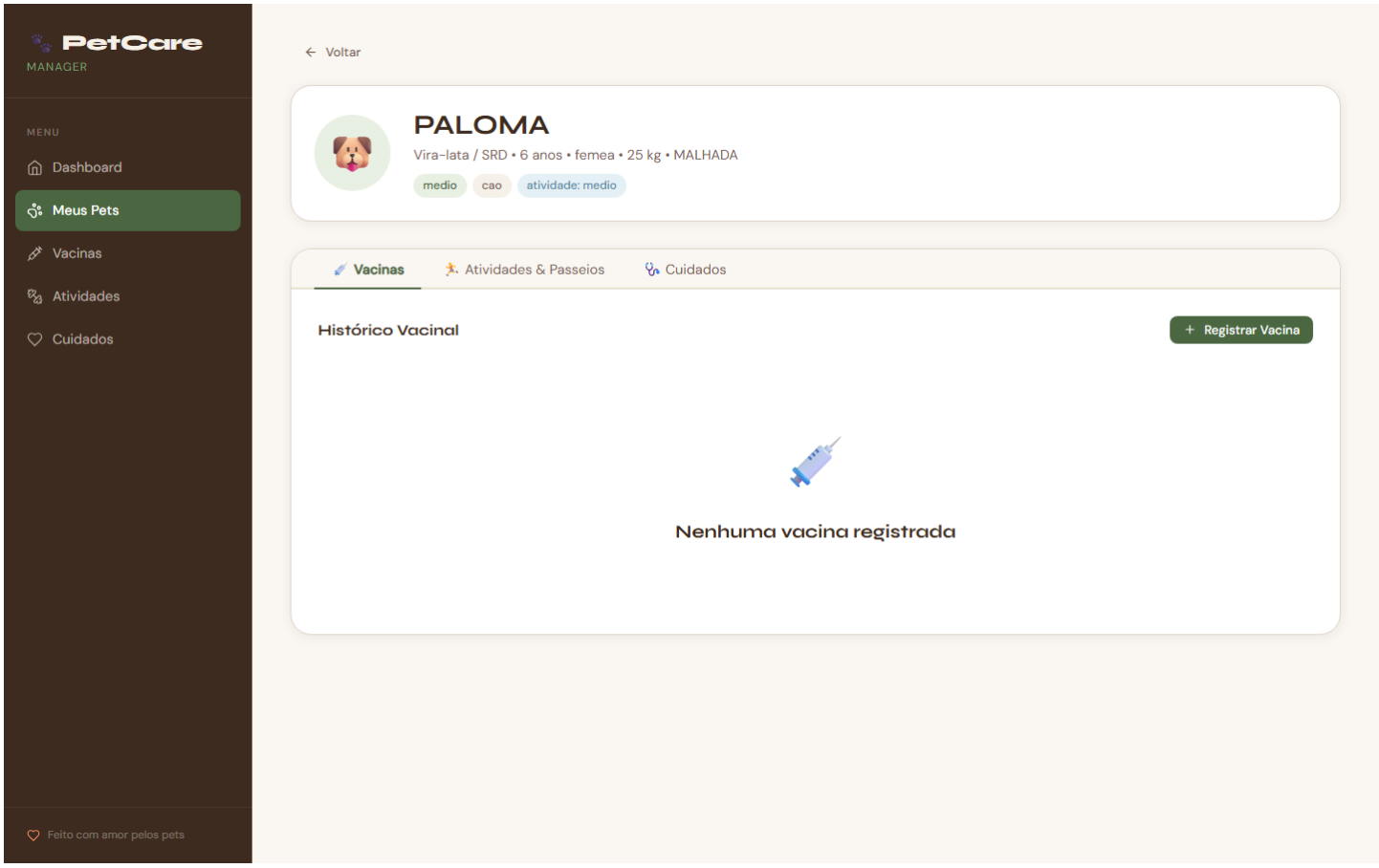
Dashboard - Visao geral com stats e alertas de vacinas



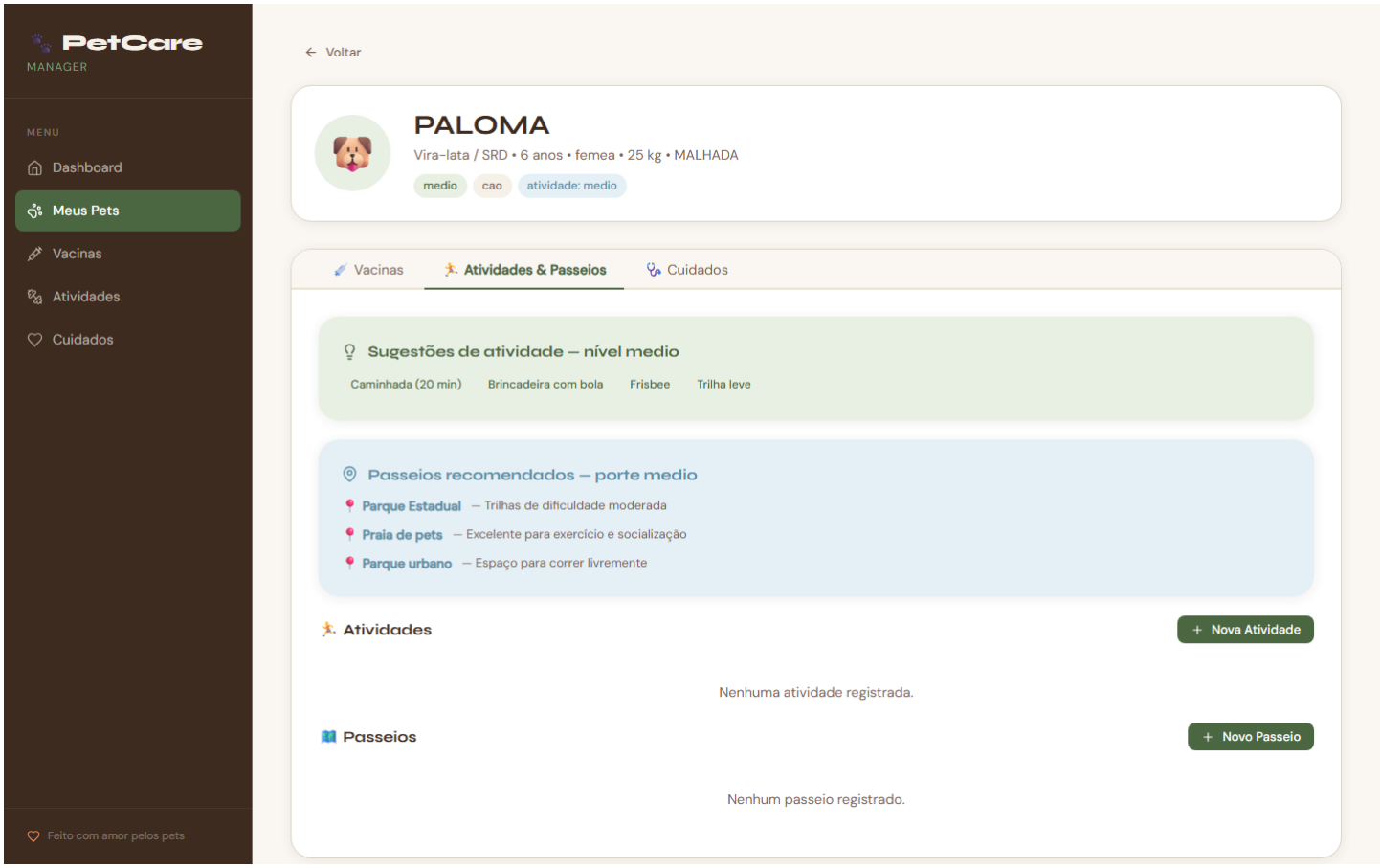
Meus Pets - Listagem com busca e cadastro



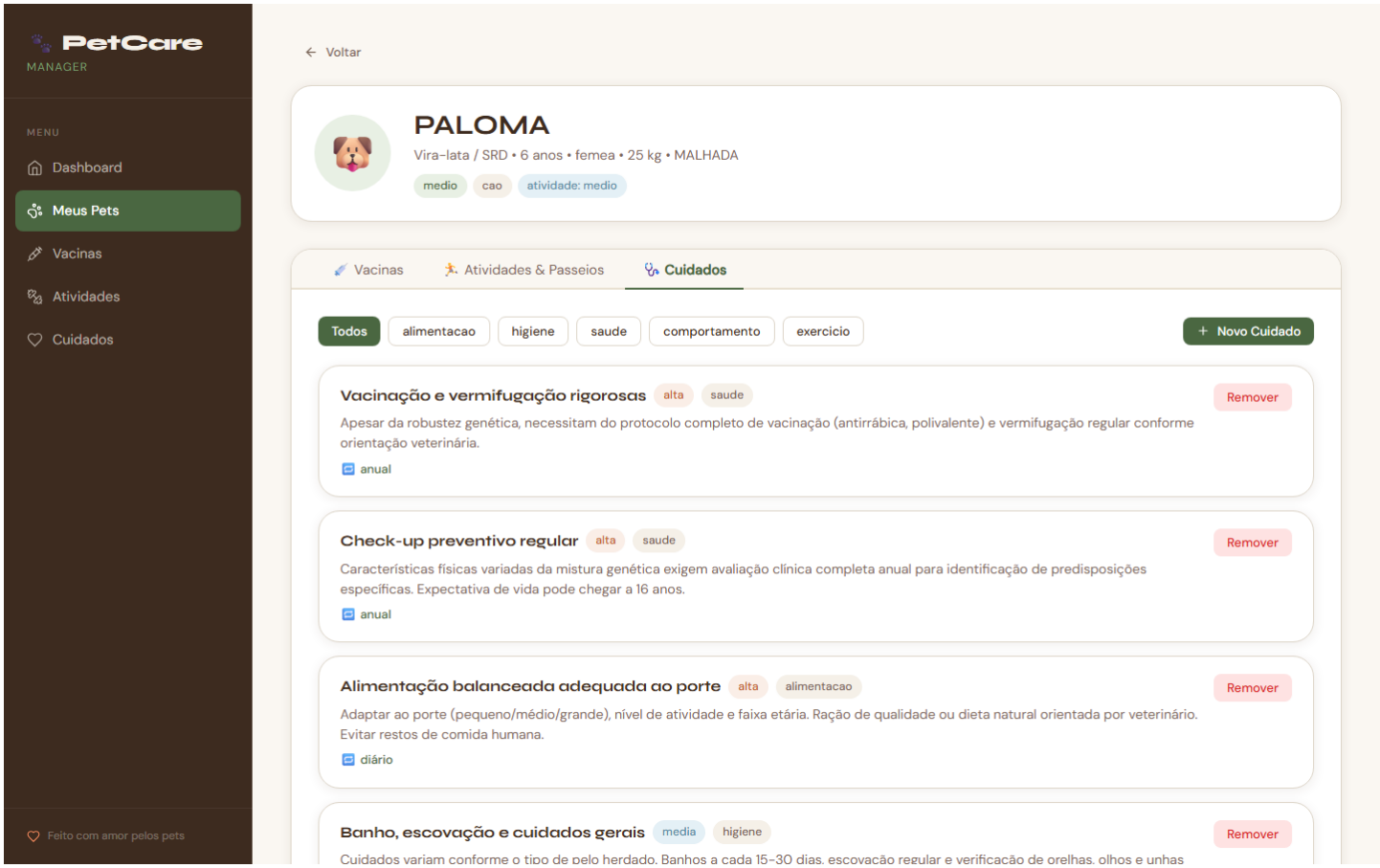
Detalhe do Pet - Aba Vacinas (historico vacinal)



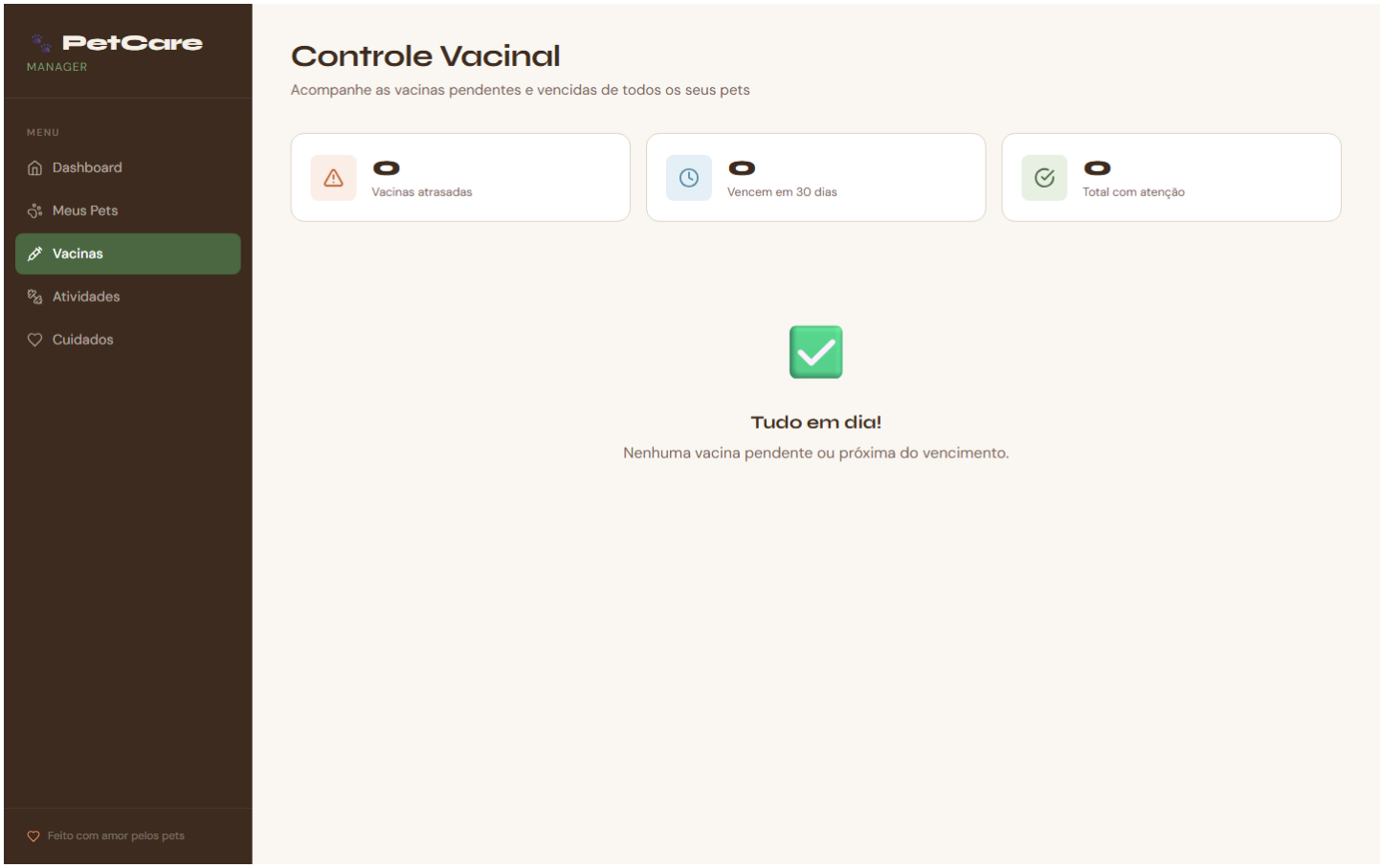
Detalhe do Pet - Aba Atividades & Passeios



Detalhe do Pet - Aba Cuidados



Controle Vacinal - Vacinas atrasadas e proximas



Atividades - Selecao de pet

Guia de Cuidados - Cuidados por raca

Swagger - Documentacao interativa da API

PetCare Manager API 1.0.0 OAS 3.1

/openapi.json

```
## 🐾 API de Gestão de Pets

Sistema completo para gerenciamento de animais de estimação.

### Funcionalidades:
- **Pets**: Cadastro e gestão de animais
- **Raças**: Informações por raça com cuidados específicos
- **Vacinas**: Calendário vacinal e controle de imunização
- **Atividades**: Registro e recomendação de atividades físicas
- **Passeios**: Sugestões de locais e roteiros
- **Cuidados**: Guia personalizado por raça
```

Raças

GET	/api/v1/racas/	Listar Racas
POST	/api/v1/racas/	Criar Raca
GET	/api/v1/racas/{raca_id}	Buscar Raca
PUT	/api/v1/racas/{raca_id}	Atualizar Raca

Pets

GET	/api/v1/pets/	Listar Pets
POST	/api/v1/pets/	Criar Pet
GET	/api/v1/pets/{pet_id}	Buscar Pet
PUT	/api/v1/pets/{pet_id}	Atualizar Pet

7. Como Executar

Pre-requisitos

- Docker e Docker Compose instalados
- Portas 3000, 5432 e 8000 disponiveis

Subir a aplicacao

```
cd petcare
docker compose up --build
```

Acessar

- Frontend: http://localhost:3000
- API / Swagger: http://localhost:8000/docs
- Banco: localhost:5432 (petcare_user / petcare_secret)

Executar testes

```
docker compose exec api pytest -v
```

52 testes automatizados usando SQLite in-memory (nao requer PostgreSQL).

Parar a aplicacao

```
docker compose down       # mantem dados
docker compose down -v    # apaga volume do banco
```